

Problem 1: The ATM Simulator (State & Memory)

Description: Write a program that simulates a simple ATM machine. The program should display a menu using a **do-while** loop:

1. Deposit
2. Withdraw
3. Check Balance
4. Exit. Use a **switch** statement to handle the user's choice.

Requirements:

- Create a function **void processTransaction(float *balance, int type)** that uses Call by Reference to update the account balance.
- Inside this function, declare a **static** variable (Storage Duration) to count the total number of valid transactions made during the session.
- Print the total number of transactions before the program exits.

Concepts Covered: Variables, Data Types, I/O, Conditionals (switch), Loops (do-while), Functions, Storage Duration (**static**), Call by Reference.

Problem 2: The String Manipulator (Pointers & Strings)

Description: Write a program that asks the user to input a single word (String). Write a function **void modifyString(char *str)** that modifies this string directly in memory. **Rule: You are NOT allowed to use array indexing (like `str[i]`).** You must strictly use Pointer Arithmetic (e.g., ***str**, **str++**).

Requirements:

- Traverse the string using a pointer.
- Count the number of vowels (a, e, i, o, u) using an **if-else** statement.
- Convert any lowercase letter to uppercase using basic math/ASCII operators (e.g., ***str = *str - 32;**).
- Print the modified string and the vowel count in the **main** function to prove the concept of **Scope** and **Pointers Basics**.

Concepts Covered: Strings, Pointers Basics, Pointer Arithmetic, Operators, Conditionals, Loops (while), Scope.

Problem 3: Matrix Diagonal Analyzer (2D Arrays & Pointers Relationship)

Description:

Write a program that asks the user to fill a 3×3 matrix with integers.

Requirements:

- Use **Nested Loops** (for loops) to take input from the user and print the matrix in a grid format.
- Write a function `int sumMainDiagonal(int matrix[3][3])` to calculate the sum of the main diagonal elements.
- Write another function `void searchElement(int *ptr, int target)` that takes the matrix as a flattened pointer (1D view of the 2D array) and searches for a specific number. Use **Arrays & Pointers Relationship** to find the number and print its location.

Concepts Covered: Multi-dimensional Arrays, Nested Loops, Arrays & Pointers Relationship, Pass by Value, I/O.

Problem 4: Array Sorter & Swapper (1D Arrays & Nested Loops)

Description:

Write a program that takes exactly 5 integers from the user and stores them in a 1D Array. Your task is to sort this array in ascending order using the Bubble Sort algorithm.

Requirements:

- Use Nested Loops to iterate through the array for sorting.
- Create a helper function `void swap(int *a, int *b)` that takes two pointers and swaps their values (Call by Reference).
- Call this `swap` function inside your sorting loops whenever two adjacent numbers are in the wrong order.
- Print the array before and after sorting.

Concepts Covered: Arrays (1D), Nested Loops, Functions, Call by Reference, Pointers Basics, Conditionals.

Problem 5: The Recursive Math Engine (Recursion & Scope)

Description:

Write a math program that uses Recursion Basics (functions calling themselves) instead of standard loops to perform calculations.

Requirements:

- Write a recursive function `int power(int base, int exponent)` to calculate the base raised to the exponent (e.g., $2^3 = 8$). Use Call by Value.
- Write a second recursive function `int sumDigits(int n)` that takes a number and sums its digits (e.g., `123` becomes $1 + 2 + 3 = 6$). Use the modulus `%` and division `/` Operators.
- Test both functions in the `main` function using user inputs.

Concepts Covered: Recursion Basics, Functions, Call by Value, Operators, Data Types.